



university of
 groningen

Potential-Based Reward Shaping for Planar Pushing

Single-Agent Control vs. Asymmetric Self-Play

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026

1. Introduction & Research Questions

Outline



1. Introduction & Research Questions

2. Background & Related Work

reward shaping · curriculum & self-play · push mechanics & SE(2)

3. Problem Definition & MDP

4. Implementation

six models: PPO-Baseline, PPO-PBRS,
PPO-Curriculum, ASP-dPose, TASP-dPose,
TASP-dPose-BP

5. Evaluation

30 held-out scenes · 20 trials · Isaac vs gym

6. Results

7. Discussion

8. Conclusion & Takeaways

Research Questions



Three Research Questions

1. **RQ1:** Can potential-based reward shaping improve the efficiency of hand-tuned dense rewards for goal reaching tasks in a robotic environment?
2. **RQ2:** Given a well-shaped PBRS reward function does adding a curriculum or asymmetric self-play improve final task success over a direct single-agent approach?
3. **RQ3:** Can a sufficiently informative reward (PBRS) *substitute* for an explicit curriculum in contact-rich pushing?

2. Background & Related Work



Related Work – Reward Shaping (PBRs)

Potential-based reward shaping :

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_F$$

- **Policy invariance:** $Q'(s, a) = Q(s, a) - \Phi(s)$ – the optimal policy is unchanged for *any* state potential Φ .
- **Episodic validity :** $\gamma_{\text{shaping}}=1$ is safe – $\sum F = \Phi(s_T) - \Phi(s_0)$ is bounded.
- **Dynamic / arbitrary potentials** – basis for our exponential Φ .

Why it matters here: lets us design a dense, well-behaved reward *without* changing the task's optimum – the foundation of every model in this thesis.

Related Work – Curriculum & Asymmetric Self-Play

Forced curricula

- **Surveys**
- **Reverse / goal** : start near the goal, expand outward
- **Precision** : gradually tighten the success tolerance

Our use: a forced 3-phase pos→rot curriculum (PPO-Curriculum)

Automatic (self-play)

- **ASP** : Alice/Bob adversarial game + ABC
- **Time-based Alice** : $R_A \propto t_B - t_A$, self-regulating
- **Regret-based**

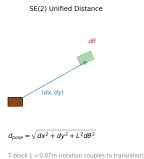
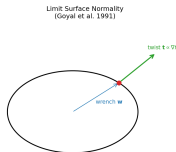
Our use: ASP-dPose (outcome) and TASP-dPose (time-based)

Related Work – Geometry & Mechanics of Planar Pushing



Push mechanics

- **Quasi-static pushing:** motion set by the friction limit surface.
- **Limit-surface normality:** every push couples translation and rotation.



SE(2) geometry

- **Unified distance:** a single scalar that combines position and orientation error, weighted by the object's physical size L
- **Equivariant manipulation:** exploiting SE(2)/SO(2) symmetry.

3. Problem Definition & MDP

Planar-Pushing MDP – State, Action & Transition



State $s \in \mathbb{R}^{28}$: EE pose (6) | object pose (14) | goal pose (6) | errors (2)

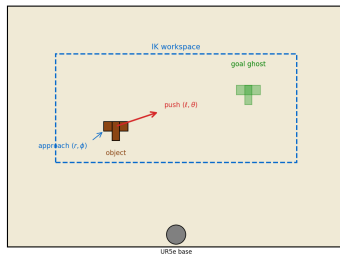
Action $a \in \mathbb{R}^4$ (object-relative): (r, ϕ, ℓ, θ) – 21 bins/dim, 194 481 discrete actions

Transition : $t \propto \nabla f(\mathbf{w})$ – limit-surface normality couples **every** push with rotation

Success: $\text{pos} < 5 \text{ cm}$ **and** $\text{rot} < 0.2 \text{ rad}$.

Validation: 30 scenes $\times$ 20 trials $\times$ up to 30 pushes

Planar Pushing Task — Object-Relative Action (r, ϕ, ℓ, θ)





Why the Fractional Reward Failed

Fractional improvement formula:

$$R = \underbrace{\alpha \cdot \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}}}_{\text{improvement}} - \underbrace{\beta_{\text{pos}} \cdot d_{\text{now}}}_{\text{pos penalty}} - \underbrace{\beta_{\text{rot}} \cdot y_{\text{now}}}_{\text{rot penalty}}, \quad \alpha=3.0, \beta_{\text{pos}}=0.5, \beta_{\text{rot}}=0.25$$

Worked example – pushing toward a goal 30 cm away, 1 cm improvement:

$$R = 3.0 \cdot 0.01 / 0.30 - 0.5 \cdot 0.29 - 0.25 \cdot 0.5 = -0.17$$

Problem 1 – Negative reward for progress: the penalty terms dominate the improvement.

Problem 2 – Near-zero variance: 99% of pushes produce $R \in [-0.5, +0.1]$.



How PBRS Fixes the Reward

PBRS potential: $\Phi(s) = w \cdot e^{-k \cdot d^2}$, $k=30$, $w=10$

Same push, same 1 cm improvement:

$$R = w \cdot [e^{-k \cdot d_{\text{now}}^2} - e^{-k \cdot d_{\text{prev}}^2}] = 10 \cdot [e^{-30 \cdot 0.29^2} - e^{-30 \cdot 0.30^2}] = +0.133$$

Why this matters:

- **Correctly signed** – every push toward the goal gets positive reward, away gets negative
- **Bounded** – $\Phi(s) \in [0, w]$, so $|R| \leq w$ regardless of distance or physics glitches
- **Markovian** – depends only on current state s , not on history or d_{prev}
- **Policy-invariant** – the optimal policy under PBRS is identical to the original task

4. Implementation



Baseline – Fractional Reward at Scale

Same fractional reward, same PPO+LSTM, same 2,048 envs, same 100K iter budget:

Model	Scene SR	Pos-only	Pos+rot
PPO single-agent	80.0%	100%	70%
ASP (Alice+Bob)	0.0%	0%	0%

Same PPO+LSTM, same push primitive, same training budget – only the architecture differs.

The adversarial goal distribution, not the reward formula, is the bottleneck. *(See Appendix K for training budget comparison.)*

All Models at a Glance



Model	Idea	Reward / curriculum
PPO-Baseline	single agent, original reward	fractional improvement (non-PBRS)
PPO-PBRS	single agent, no curriculum	two-potential PBRS (pos, rot)
PPO-Curriculum	single agent + staged curriculum	PBRS + forced pos→rot
ASP-dPose	self-play (Alice/Bob)	SE(2) PBRS, outcome Alice (+5/ - 1)
TASP-dPose	self-play (Alice/Bob)	SE(2) PBRS, time-based Alice
TASP-dPose-BP	self-play + Bob penalty	SE(2) PBRS, full Sukhbaatar symmetric

All six models share the same PPO+LSTM backbone and object-relative push primitive.

[Schulman et al. 2017] [Ng et al. 1999] [Sukhbaatar et al. 2018]

PPO-PBRS – Single-Agent (Recommended Baseline)



Architecture: Single PPO agent, LSTM(28→256) + MLP[512, 256] trunk, MultiCategorical head (4 dims $\times$ 21 bins). No curriculum, no Alice/Bob, no GoalEncoder.

Training: 528 envs, 15 pushes/rollout, ~ 3000 iterations ~ 1.6 days wall-clock (~ 0.022 it/s).

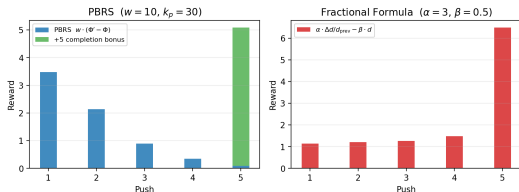
Why it works:

1. **Dense, bounded gradient** – PBRS fires on every push
2. **Fixed goal distribution** – stationary random goals; no adversarial shift
3. **Single network** – one agent, one optimizer; no multi-agent instability



PBRS vs Fractional– Reward Signal Analysis

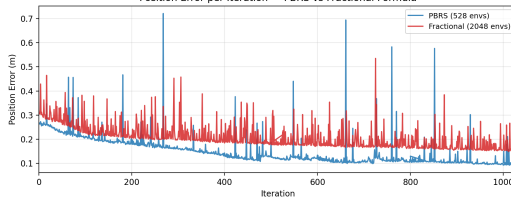
Simulated 5-Push Episode — PBRS vs Fractional Formula



Per-push reward – Fractional vs PBRS: same 5-push episode. Fractional formula: negative or zero for 80% of pushes. PBRS: correctly-signed, bounded on every push.

Dense signal on every push, bounded rewards, stable value – PBRS avoids the gradient starvation and critic instability of the old fractional formula.

Position Error per Iteration — PBRS vs Fractional Formula



Position error per iteration (TensorBoard): PBRS at 528 envs converges to lower PosErr than fractional formula at 2,048 envs – 4× more environment-efficient.

Why PBRs for ASP? – Decoupling Advantage from Value



GAE advantage decomposes into two parts:

$$\hat{A}_t = \underbrace{\sum (\gamma\lambda)^l [\Phi(s_{t+l+1}) - \Phi(s_{t+l})]}_{\text{always correctly-signed}} + \underbrace{\sum (\gamma\lambda)^l [\gamma V(s_{t+l+1}) - V(s_{t+l})]}_{\text{biased when } V \text{ is stale}}$$

Φ -term: PBRs potential difference

$\Phi(s) = w \cdot e^{-k \cdot d^2}$ increases when object moves toward goal. Correct sign on every push regardless of $V(s)$ accuracy.

V -term: value function bootstrap

Under ASP, Alice shifts goal distribution $\rightarrow V(s)$ perpetually stale. GAE bootstraps from wrong values $\rightarrow$ biased advantage.

Outcome: PBRs lifts ASP 0% $\rightarrow$ 16.7% via the Φ -term – but only partially, because the V -term bias remains.

Single-agent: no distribution shift $\rightarrow V$ accurate $\rightarrow$ both align $\rightarrow$ 80%.

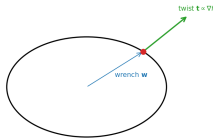
SE(2) Unified Distance – Self-Play Variants

SE(2) metric: $d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$

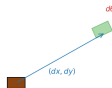
- Single scalar combining (dx, dy) and $d\theta$; L = char. length (T-block 0.07 m)
- **Single-potential PBRs:** $\Phi(s) = w e^{-k d_{\text{pose}}^2}$, $k = 30$, $w = 10$
- Eliminates competing pos/rot gradients
 - aligns reward with the coupled physics

Result: SE(2) does *not* rescue self-play – ASP-dPose reaches only 6.7% validation SR vs single-agent 80%. The self-play curriculum is the root cause, not the metric.

Limit Surface Normality
(Goyal et al. 1991)



SE(2) Unified Distance



$$d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$$

T-block $L = 0.07\text{m}$ (rotation couples to translation)

PPO-Curriculum – Position→Rotation Curriculum



Design: Same PPO agent as PPO-PBRS + 3-phase curriculum:

1. $w_{\text{rot}} = 0$, position-only training
2. Triggered when episodic position-SR ≥ 0.5 for 20 iters: w_{rot} ramps $0 \rightarrow 10$ over 200 iters
3. Full PBRS (multi-objective)

Trigger: gates on episodic position-only SR ≥ 0.5 over a 20-iter lookback. Curriculum now activates and reaches **76.7% SR** – only 3.3pp below PPO-PBRS. (See Appendix H for original trigger bug details.)

Self-Play – ASP / TASP Architecture (Alice/Bob)



Alice (Goal Proposer):

- PPO agent, free-roaming push phase – leaves T-block in a new SE(2) pose
- That pose becomes Bob's target goal
- Reward: +5 if Bob fails to reproduce it, −1 if Bob succeeds

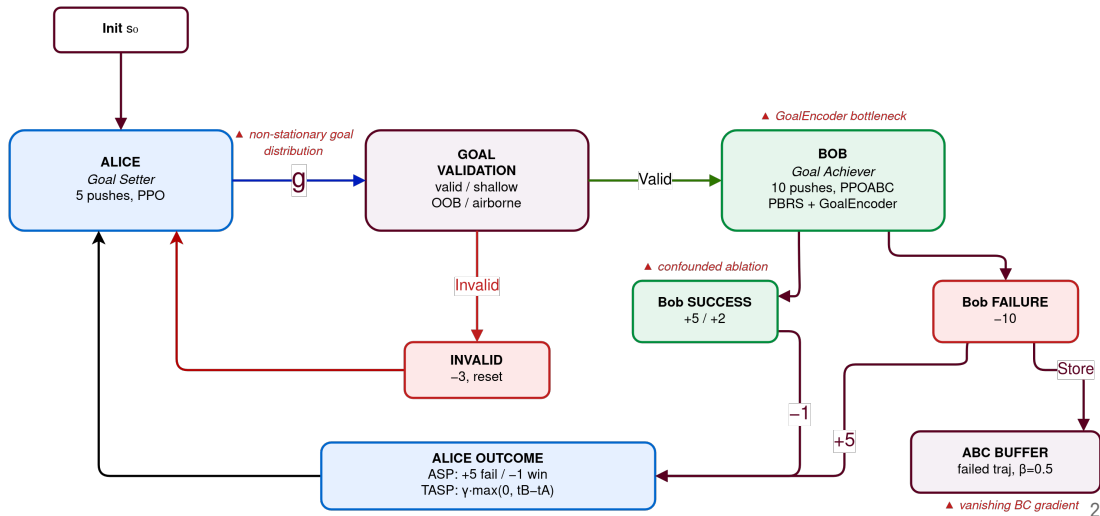
Bob (Goal Achiever):

- PPOABC + GoalEncoder – must reproduce Alice's goal pose from scratch
- Reward: PBRs dense shaping + sparse completion bonuses (+5/+2)
- Failed trajectories stored in ABC buffer for behavioral cloning ($\beta_{abc}=0.5$)

Two variants: both use the SE(2) unified d_{pose} metric (T-block); they differ only in Alice's reward – **ASP-dPose** (outcome-based) vs **TASP-dPose** (time-based).



ASP Loop – Alice $\leftrightarrow$ Bob Cycle





ASP-dPose vs TASP-dPose – Alice Reward

Both share the SE(2) d_{pose} Bob reward and T-block object; only Alice's reward differs.

Self-play variant	Alice reward	Valid SR
ASP-dPose (outcome-based)	$R_A = +5 \text{ fail} / -1 \text{ win}, \beta_{\text{abc}}=0.5$	6.7%
TASP-dPose (time-based)	$R_A = \gamma_{\text{sp}} \cdot \max(0, t_B - t_A), \text{no ABC}$	16.7%

Findings:

- Time-based Alice (TASP-dPose) outperforms outcome-based (ASP-dPose) by +10pp
- TASP-dPose disables ABC – the gain comes from the time-based reward alone
- Best self-play variant (TASP-dPose, 16.7%) still **4.8** $\times$ below single-agent (PPO-PBRS, 80.0%)

5. Evaluation

Validation Test Suite – Top 10 Held-Out Scenes



\textbf{Top row:} 5 easiest scenes (3-4 passes) | \textbf{Bottom row:} 5 hardest scenes (0 of 4 models pass)

T-block **start pose** → **goal pose**. Every model is evaluated on an identical 30-scene suite (20 trials × up to 30 pushes per scene).



Isaac Lab – the Right Venue for Self-Play

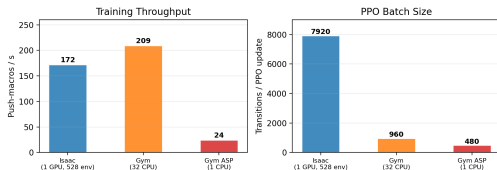
GPU-batched parallelism – 528 environments stepped simultaneously on one RTX Pro 6000. Produces 7,920-sample PPO batches – $8\times$ larger than gym-pusht (960).

Large batches stabilize PPO+LSTM gradients – essential for the non-stationary two-agent self-play objective.

Robotic fidelity – UR5e arm + 3D contact physics + cuRobo IK = the actual SE(2) task. gym-pusht is a 2D diagnostic only; it cannot test self-play under realistic contact dynamics.

Fair budget – self-play and single-agent run at identical per-iteration wall-clock (~ 0.022 it/s). Isaac gives self-play its **best shot**, matched to the winning single-agent –

Isaac vs Gym — Why GPU-Batched Parallelism Matters

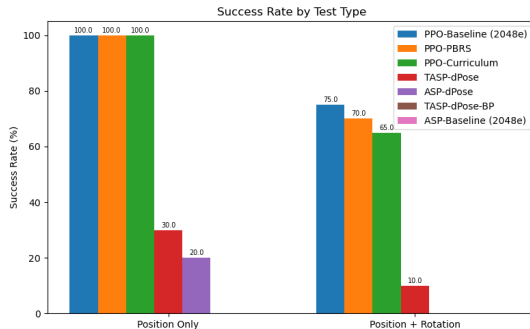
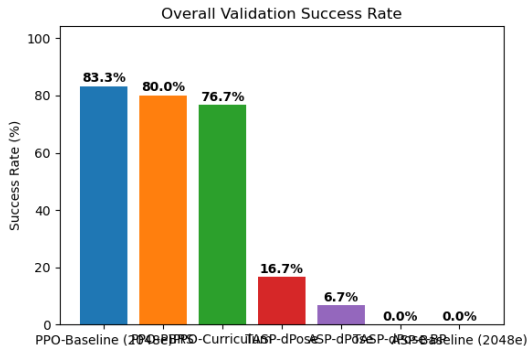


Throughput is comparable; batch size is the differentiator.

6. Results

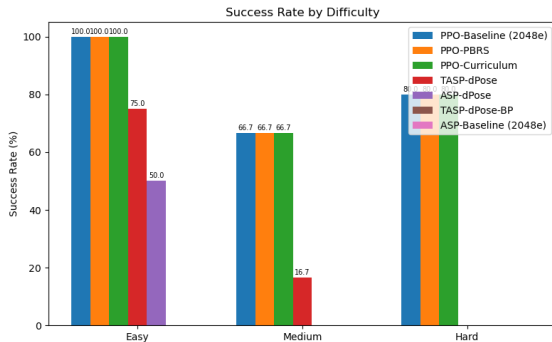


Multi-Model Validation



Overall SR & SE – 30 T-block scenes, best-of-20
PPO-PBRS vs PPO-Curriculum: 80.0% vs 76.7% – 3.3pp gap not statistically significant
($p > 0.05$ on 30 independent scenes). **Self-play (ASP-dPose, TASP-dPose): 4.8–11.9 $\times$ below**

Comparison Summary – All Models with Error Bars



Key numbers (30 T-block scenes):

Model	SR	SE	95% CI
PPO-Baseline	80.0%	7.3%	65–95%
PPO-PBRS	80.0%	7.3%	65–95%
PPO-Curriculum	76.7%	7.7%	61–92%
TASP-dPose	16.7%	6.8%	3–30%
ASP-dPose	6.7%	4.6%	0–16%
TASP-dPose-BP	0.0%	–	–

PPO-PBRS vs PPO-Curriculum gap: 3.3pp – not

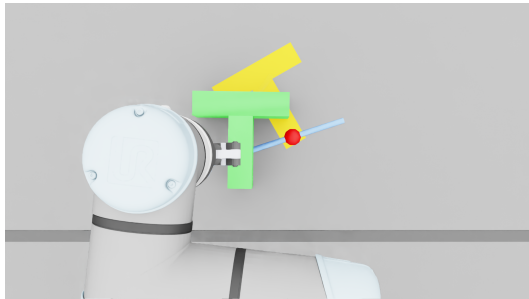
statistically significant ($p > 0.05$). Both solve the task similarly. Self-play variants are $4.8\text{--}11.9\times$ below – this gap **is** significant.

Error bars = ± 1 standard error across scenes. Base ASP and the GoalEncoder-ablation are excluded – neither has α^{29}

PPO-PBRS – Validation Results

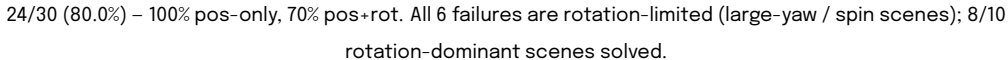
30 held-out T-block scenes (20 trials each):

- **80.0% SR** (24/30) – pos-only **100%**, pos+rot **70%**
- PosErr **0.032 m**, RotErr **0.568 rad** (mean over all 30 scenes, incl. failures)
- Per-attempt (trial-level) SR **66%**; scene SR counts a scene solved if any of its 20 trials passes



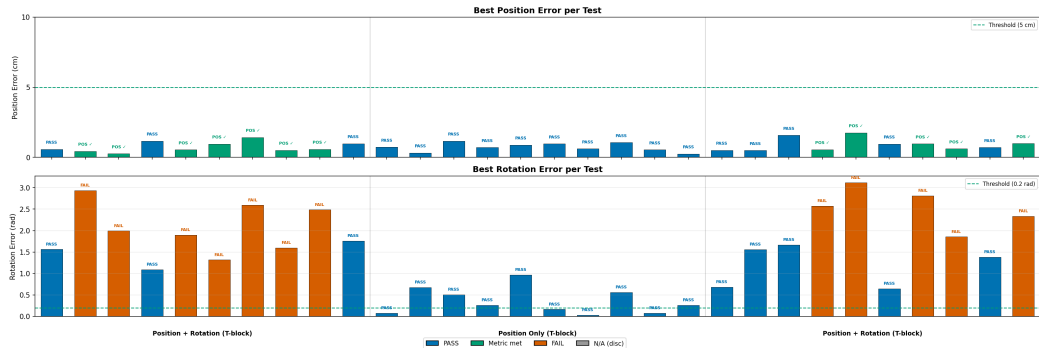
Overhead view – PPO-PBRS solving a pos+rot scene ▶

[play clip](#)





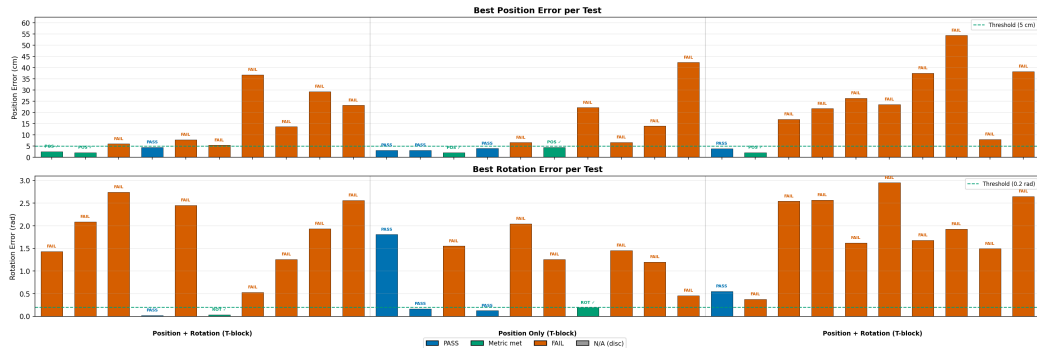
PPO-Curriculum – Per-Test Error



23/30 (76.7%) – same 100% pos-only, 65% pos+rot. Tighter position precision from Phase 1 (0.023 vs 0.032 m), but marginally worse rotation (0.663 vs 0.568 rad).



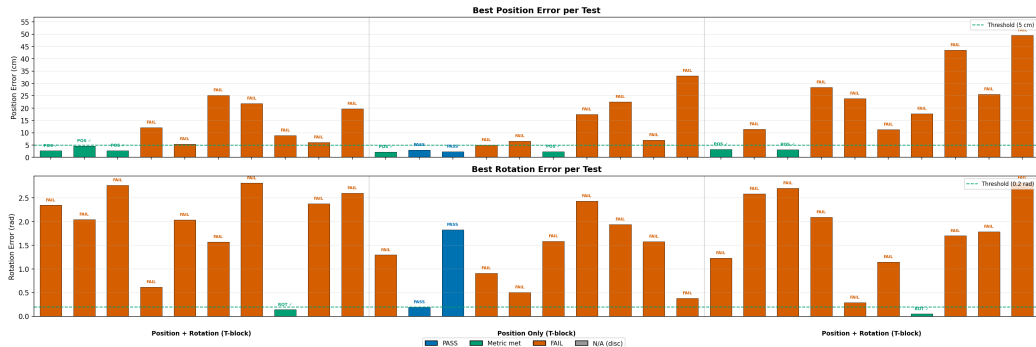
TASP-dPose – Per-Test Error



5/30 (16.7%) – 3 pos-only, 2 pos+rot. Only the easiest scenes pass, each with 1-2 successful trials; rotation never learned (RotErr 1.457 rad).

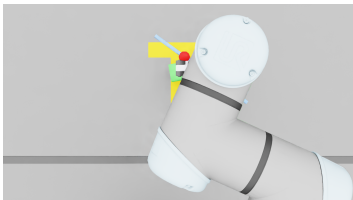


ASP-dPose – Per-Test Error

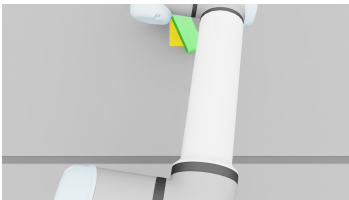


2/30 (6.7%) – only the two easiest position-only scenes pass. Zero pos+rot success; rotation never learned (RotErr 1.612 rad). Outcome-based Alice collapses entirely.

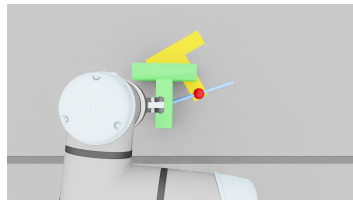
PPO-PBRS – Validation Runs (overhead)



Forward (pos-only) ▶ [play clip](#)



Lateral push (pos-only) ▶ [play clip](#)

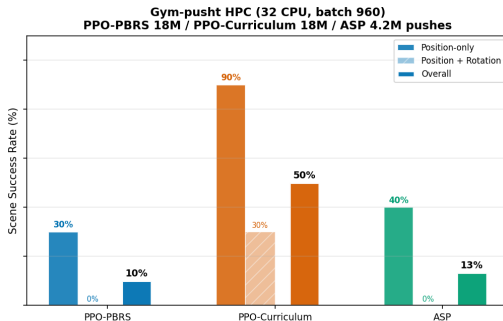
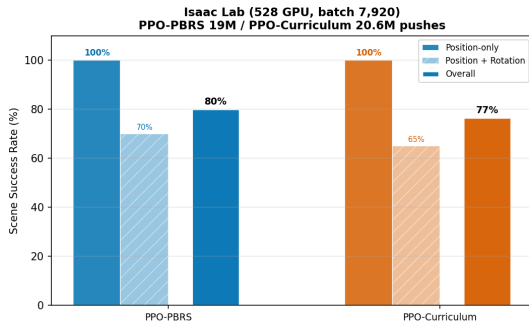


Translate + rotate ▶ [play clip](#)



Cross-Environment Validation – Isaac vs Gym

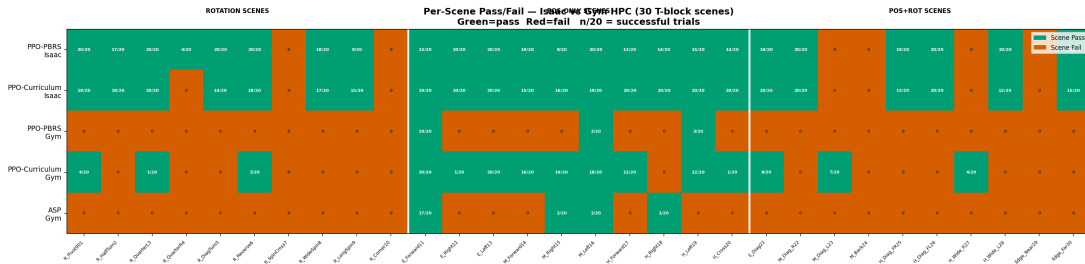
Cross-Environment Validation SR — Isaac Lab vs Gym-pusht HPC
Same thesis gate, same 30 T-block scenes, same total push budget (~18-20M for PPO-PBRS / PPO-Curriculum)



The ordering single-agent $\approx$ curriculum $\gg$ self-play holds in both simulators. Gaps compress in the simpler gym env: PPO-Curriculum benefits most from the small-batch regime (50% gym vs 76.7% Isaac), consistent with the RQ3 batch-size caveat.



Per-Scene Outcomes – Isaac vs Gym



Green = scene solved (any of 20 trials), red = failed; cell text = successful trials / 20. The hardest pos+rot scenes (right block) defeat every configuration except the two single-agent models on Isaac. Self-play (gym) solves only a few easy position-only scenes.

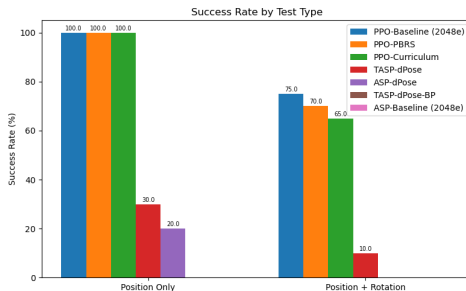
7. Discussion

Discussion – What the Validation Suite Reveals

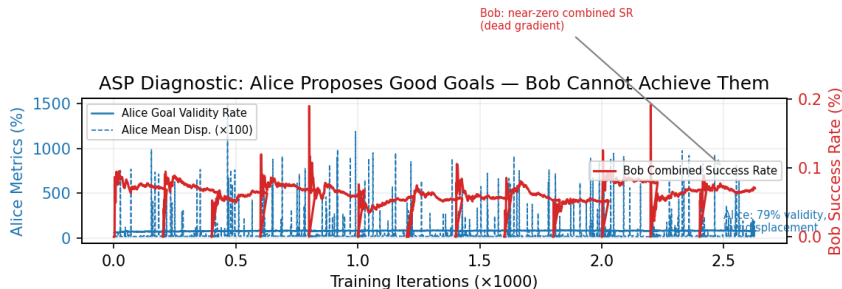


The combined pos+rot gate is the wall:

- Position-only scenes (T11–T20): solved by both single-agent models (100%).
- Pos+rot scenes (T1–T10, T21–T30): the bottleneck – PPO-PBRS 70%, self-play $\leq 10\%$.
- Failures are almost always **rotation**-limited, not position – consistent with the coupled limit-surface physics.
- Hardest scenes are large-yaw / wide-translation (e.g. R_SpinCross, H_Wide): they need many coordinated



ASP Diagnostic – Alice vs. Bob



Alice proposes valid, low-displacement goals, yet Bob cannot achieve the combined pos+rot objective under the non-stationary adversarial goal distribution.



Why ASP Fails – Failure Modes

1. **Reward signal starvation (affects both single-agent and ASP):**

- Original fractional formula gives ≈ 0 reward for 99% of pushes – GAE sees noise
- Sparse bonuses (+5/+2) fire on $\sim 1\%$ of pushes – too few events for stable gradient
- PBRs provides correctly-signed reward on *every* push: +0.02 to +0.50 per step
- Single-agent: PBRs enables 80% SR. ASP: PBRs lifts 0% $\rightarrow$ 16.7%

2. **Non-stationary goal distribution (ASP-specific):**

- Alice updates every iteration $\rightarrow$ Bob's training distribution shifts
- Bob's value function $V(s)$ is trained on old goals, evaluated on new goals – systematically biased
- GAE bootstraps with stale $V - \hat{A}_t$ sign is wrong on most transitions
- PPO clips $\pi_{\text{new}}/\pi_{\text{old}}$ – gradient killed when ratio moves outside $[1 - \epsilon, 1 + \epsilon]$
- Single-agent has fixed random goals – no distribution shift, V stays accurate, 80% SR



Why ASP Fails – The Evidence

What we observe across 5 ASP configurations:

- Original fractional reward, 2,048 envs: 0%
- PBRs outcome-based (ASP-dPose): 6.7%
- PBRs time-based (TASP-dPose): 16.7% (best variant)
- PBRs time-based + Bob penalty: 0%
- gym-pusht (different sim, same gate): 13.3%
- Single-agent ceiling (PPO-PBRs): 80.0%

Why this is not just a budget issue:

- All models trained for 3000 iters at 528 envs on identical hardware
- ASP and single-agent have identical per-iteration wall-clock (~ 0.022 it/s)
- The gap is **structural**, not due to compute starvation

Combined effect: The non-stationary goal distribution + multi-axis design



Caveat – Confounded Ablation

Problem: the self-play variants differ from single-agent PPO-PBRS on *at least 4 axes simultaneously*:

Variable	PPO-PBRS (80.0%)	Self-play (6.7–16.7%)
Agent count	1	2 (Alice + Bob)
Goal distribution	Fixed random	Adversarial, shifting
GoalEncoder	None	ϕ -MLP 8D latent
ABC buffer	None	Yes (ASP-dPose) / No (TASP-dPose)
Historical pool	None	Yes
Episode budget	15 pushes/agent	5 (Alice) + 10 (Bob)
PPO updates/iter	1	2 (Alice, then Bob)

The $4.8\times$ gap cannot be attributed to any single lever.

- TASP-dPose (time-based Alice, no ABC) vs ASP-dPose (outcome-based, ABC enabled) isolates the Alice reward axis – TASP-dPose gains +10pp
- But this is a *single* 1-axis experiment within a 7-axis package change



Key Result – PBRs Substitutes for Curriculum

Model	Valid SR	vs Single-Agent
PPO-PBRs (single-agent)	80.0%	– (baseline)
PPO-Curriculum	76.7%	–3.3pp (n.s.)
TASP-dPose (time-based self-play)	16.7%	4.8× gap
ASP-dPose (outcome self-play)	6.7%	11.9× gap

The Substitution Finding (RQ3)

- **Curriculum adds no value at this scale:** PPO-Curriculum (76.7%) trails PPO-PBRs (80.0%) by 3.3pp, but this gap is **not statistically significant** ($p > 0.05$). A sufficiently informative reward (PBRs) *substitutes* for the staged curriculum.
- **Scope / caveat:** substitution holds when the PPO batch is large enough for stable gradients (Isaac, batch 7,920). At small batch (gym, batch 960) the curriculum *does* help.
– PPO-Curriculum 50% vs PPO-PBRs 10%. So the substitution is regime-dependent.



What We Found – Theory vs Practice

Self-play did *not* yield a useful curriculum in this setting

In this contact-rich, IK-gated, multi-objective SE(2) pushing task under a single-GPU budget, our self-play implementation produced 6.7–16.7% validation SR vs 80.0% for single-agent. The adversarial mechanism did not generate a learnable progression of goals. This does *not* refute self-play in general – it demonstrates a failure mode under realistic robotics constraints.

The bottleneck is the goal *distribution*, not the goal *representation*

Bob's individual position and rotation skills are partly learned, but the combined gate rarely fires under the non-stationary adversarial goal distribution. The failure sits at the exploration / training-distribution level, not the observation encoding.

Consistent across reward schemes, simulators, and ASP variants



What We Found – Theory vs Practice

Forced curriculum: functional but not beneficial

PPO-Curriculum's original trigger (per-push mean PosErr $< 0.08m$) was mis-specified – the threshold was unreachable by construction. A corrected trigger (episodic position-SR ≥ 0.5) produces a competent model (76.7%), but the staged pos $\rightarrow$ rot curriculum does *not* beat the simpler no-curriculum baseline (80.0%). Curriculum works – it just doesn't add value at this batch size.

PBRs theory strongly supported

The policy invariance theorem holds in practice, even in contact-rich manipulation with stochastic PhysX physics. PBRs provides clean, bounded, Markovian rewards. Parameters $k_p=30$, $k_r=5$, $w=10.0$ are robust across all single-agent runs.

8. Conclusion & Takeaways

Conclusion



1. PBRs enables on-policy learning for planar pushing (RQ1)

PPO with the original fractional reward requires 2,048 envs to reach 80%. PBRs achieves identical performance at 528 envs – $4\times$ fewer – by providing dense, correctly-signed, bounded rewards on *every* push. PBRs preserves the optimal policy while making the reward tractable for on-policy GAE.

2. Self-play fails structurally on this specific task (RQ2)

Across five ASP variants – original fractional reward (0%), outcome-based PBRs (6.7%), time-based PBRs (16.7%), symmetric Bob-penalty PBRs (0%), and a gym-pusht implementation (13.3%) – **no self-play configuration exceeds 17%**, against an 80% single-agent ceiling. The adversarial goal distribution creates a perpetually stale value function that prevents stable on-policy gradient estimates.

Conclusion



3. PBRs can substitute for a curriculum (RQ3)

A well-shaped reward makes the staged curriculum redundant at sufficient batch size: PPO-Curriculum (76.7%) is statistically tied with PPO-PBRs (80.0%). The substitution is regime-dependent – at small PPO batch the curriculum does help (gym: 50% vs 10%). Self-play (6.7–16.7%) helps in neither regime.

Start simple. PPO-PBRs single-agent at 528 envs trains in ~ 1.6 days on an RTX Pro 6000 and achieves 80% SR. PBRs provides the principled reward foundation; the training-goal distribution is a separate, lower-leverage knob. Add complexity only when proven to help – and expect it usually won't.



Limitations & Research Gaps

What we couldn't test (training budget limited to 3000 iters $\times$ 528 envs):

- **TASP-dPose-BobPenalty tested:** Bob time penalty (full Sukhbaatar symmetric reward) collapsed to 0% – penalty applied on catastrophe phase ends created a perverse “fail-fast” equilibrium. Asymmetric reward structure (Alice incentivised, Bob unpenalised) proved necessary.
- **Time-based self-play + ABC:** time-based Alice with ABC enabled ($\beta_{abc} = 0.5$). ABC might boot after Alice converges
- Larger models (Transformer, diffusion policy), sim-to-real transfer (physical UR5e)

Remaining literature gaps (from systematic survey of 81 papers):

- No paper on SE(2) metric design specifically for RL reward shaping
- No paper on exploiting continuous rotational symmetry (SO(2)) in object manipulation RL
- No formal convergence analysis of effort-based self-play rewards



Future Work – Promising Directions

1. **Zero-sum time-based self-play (TASP-dPose-BobPenalty):** Bob penalty creates near-zero-sum game. From differentiable game theory, zero-sum games have convergent (Hamiltonian) dynamics – this could stabilize self-play where general-sum diverges
2. **ABC after Alice convergence:** Currently ABC fails because Alice acts randomly ($bc_ratio \approx 1.0$). If Alice converges first, Bob starts with a meaningful imitation target
3. **Relaxed combined success gate:** The combined pos+rot gate is the bottleneck (70% single-agent vs 0-10% for self-play). Partial credit for one objective could provide gradient to learn both
4. **PAIRED :** Antagonist minimizes protagonist's regret rather than maximizing difficulty – solvable but challenging environments, addressing the exact failure mode

Computational Overhead – Self-Play Is Not Slower

Controlled measurement (same machine, same 528 envs, same cuRobo IK + physics):

PPO-PBRS (single-agent) vs the self-play variants run at $\sim 1.0\times$ per-iteration wall-clock (~ 0.022 it/s for both). The 2-agent machinery (Alice + Bob, GoalEncoder, ABC, historical pool) adds **no** measurable per-iteration overhead – the shared cuRobo IK + PhysX contact step dominates the cost.

Key result:

- Self-play is **not** slower per iteration – it simply **does not learn**
- The real cost of self-play is model/code complexity and added failure modes (two networks, non-stationary objective, ABC), *not* throughput
- So the $4.8\times$ performance gap is **not** a compute-budget artifact – both used the same compute

Extra Takeaway – ManagerBasedRL vs DirectRL



An engineering observation, separate from the scientific claims.

- Isaac Lab's **ManagerBasedRLEnv** (Action/Observation/Event/Termination managers) adds per-step Python dispatch; **DirectRLEnv** bypasses it with direct tensor ops.
- This is an *architecture* cost, independent of ASP – self-play and single-agent run at the same per-iteration time ($\sim 1.0\times$).

API	env-steps/s	relative
DirectRLEnv	[to measure]	[run pending]
ManagerBasedRLEnv	[to measure]	[run pending]

Controlled same-task measurement to be run; numbers will replace the placeholders. No multiplier is claimed until then.



university of
 groningen

Thank You – Questions?

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026



Appendix Outline

- A. Push Mechanics – Limit Surface & Characteristic Length L
- B. Training curves for all PBRs models (Success Rate, Reward, Loss)
- C. PBRs parameter sensitivity (k_p , k_r , w_{pos} , w_{rot})
- D. Validation scene descriptions (30 test cases)
- E. cuRobo IK pipeline and workspace configuration
- F. Observation space specifications (all model variants)
- G. Per-test error bars – PPO-PBRs vs PPO-Curriculum head-to-head and all-model comparison (95% binomial CIs)
- **H. P82 Curriculum Trigger Bug** – why the original trigger never fired
- **I. BobPenalty Fail-Fast Equilibrium** – degeneracy mechanism
- **J. Confounded Ablation Disaggregation** – 7-axis variation table
- **K. Training Budget Comparison** – envs, iterations, push-macros
- **L. Validation Protocol** – scene set, trials, disclosed limitations



PBRS – The Shaping Theorem

Ng et al.:

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_{\text{shaping reward } F}$$

Policy invariance: $Q'(s, a) = Q(s, a) - \Phi(s)$ – optimal actions unchanged.

Our potential functions (PPO-PBRS / PPO-Curriculum):

$$\Phi(s) = w_{\text{pos}} e^{-k_p d_{\text{pos}}^2} + w_{\text{rot}} e^{-k_r c(s)}, \quad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2}$$

$k_p=30, k_r=5, w_{\text{pos}}=w_{\text{rot}}=10.0$

Dense reward: $F(s, s') = \Phi(s') - \Phi(s)$ – difference only, no constant drain.

Episodic PBRS : $\gamma_{\text{shaping}}=1.0$ valid – $\sum F = \Phi(s_T) - \Phi(s_0)$ bounded.



PBRs – Why It's the Right Choice

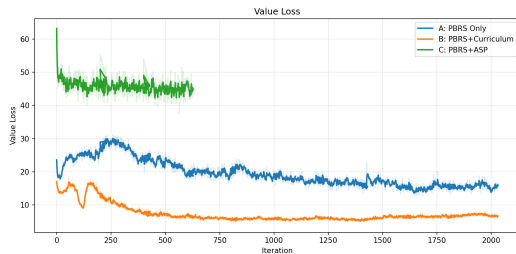
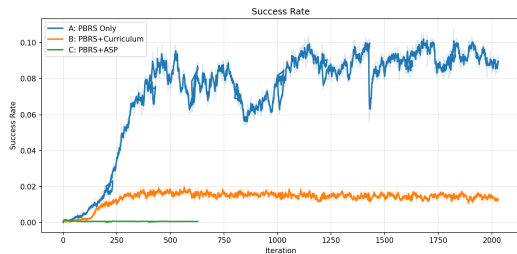
Properties of $F(s, s') = \Phi(s') - \Phi(s)$:

1. **Markovian** – depends only on s, s' , not history.
2. **Bounded** – exponential potentials prevent reward explosions from PhysX contact spikes.
3. **Zero-sum cycles** – detour pushes net to zero; agent free to explore without reward hacking.
4. **No constant drain** – $F=0$ when stationary; no penalty for hesitation or deliberation.

Key consequence: you can replace any ad-hoc reward with a potential-based one *without changing what the agent learns* – only *how fast* it learns. Reward design and policy optimisation are decoupled.



PPO-PBRS – Training Dynamics



Left: PPO-PBRS (blue) steadily improves training success rate over 3000 iterations. PPO-Curriculum (orange) follows a similar trajectory but converges slightly lower. Self-play variants remain near baseline throughout. **Right:** Stable value function – no explosion. Initial $V \approx 0.057$ with gain= 0.01.



PPO-Curriculum vs PPO-PBRS

Metric	PPO-PBRS	PPO-Curriculum	Gap
Validation SR	80.0%	76.7%	1.04×
PosErr (m)	0.032	0.023	B 28% lower
RotErr (rad)	0.568	0.663	A 14% lower
Pos-only SR	100%	100%	equal
Pos+rot SR	70%	65%	5pp

Result: Curriculum improves position precision (0.023 vs 0.032 m) from the pos-only phase, but slightly degrades rotation (0.663 vs 0.568 rad). Net effect: B is 3.3pp behind A. **Statistical note:** The 3.3pp A vs B gap is **not** significant ($p > 0.05$, two-tailed z-test; SE of difference = ± 10.6 pp across 30 independent scenes). Both A and B solve the task similarly well. **Takeaway:** A staged curriculum *can* work alongside PBRS, but does not beat the simpler no-curriculum baseline.



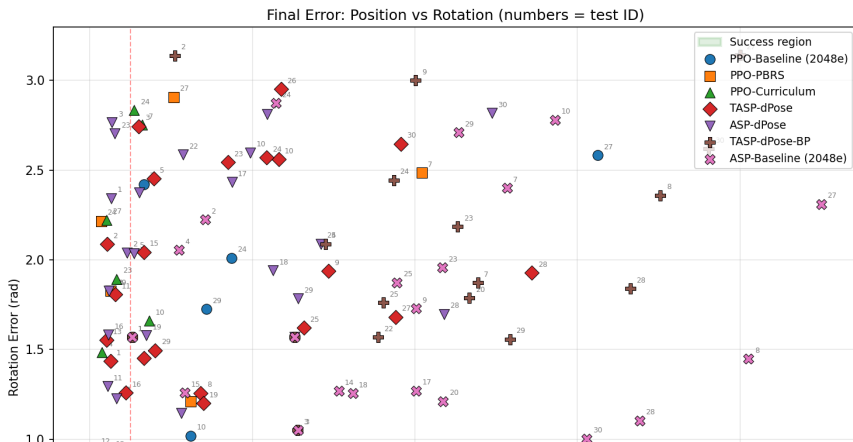
Self-Play – Validation Comparison

Model	Valid SR	PosErr	RotErr
PPO-PBRS (single-agent, ref.)	80.0%	0.032 m	0.568 rad
TASP-dPose (time-based)	16.7%	0.158 m	1.457 rad
ASP-dPose (outcome-based)	6.7%	0.143 m	1.612 rad

Key findings:

- Time-based self-play (TASP-dPose) is $2.5\times$ better than outcome-based (ASP-dPose), but still **$4.8\times$ below** single-agent PPO-PBRS
- No self-play variant achieves pos+rot SR above 10% – the combined gate remains the bottleneck
- Self-play complexity provides no benefit

Comparison Summary – Per-Scene Position × Rotation Error



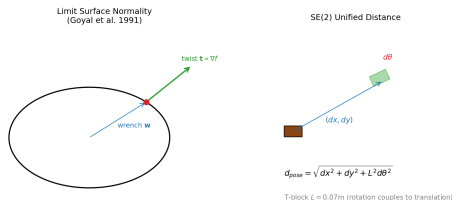
Appendix A – Push Mechanics: Limit Surface & L



Limit surface normality: $\mathbf{t} \propto \nabla f(\mathbf{w})$ – twist is normal to the surface, so every push couples translation *and* rotation.

Characteristic length L (radius of gyration):

- **T-block** $L = 0.07$ m – an off-centre push couples translation *and* rotation
- L is a physical constant, *not* a hyperparameter

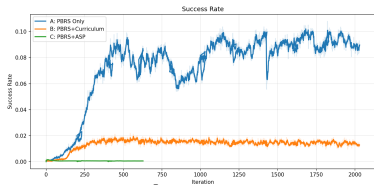


The T-block rotates under an off-centre push (coupled SE(2) motion)

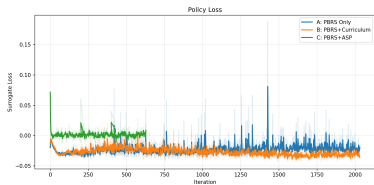
Appendix B – Training Curves (All PBRs Models)



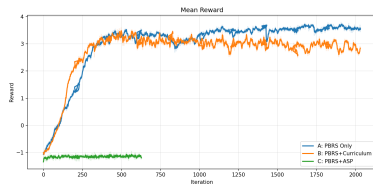
Success Rate



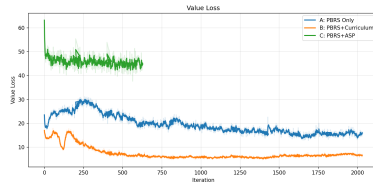
Policy Loss



Mean Reward



Value Loss

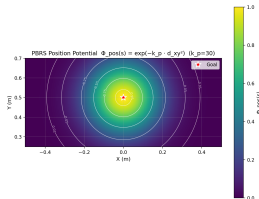


Appendix C – PBRs Theory Analysis



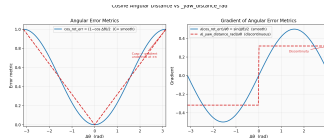
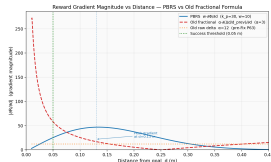
Potential Landscape

$$\Phi(s) = w \cdot \exp(-k \cdot d^2)$$



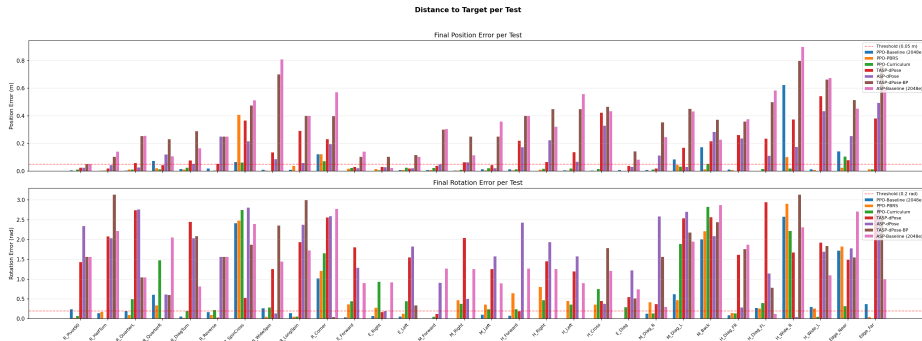
Gradient Comparison

PBRs vs Fractional Formula



Cosine Angular Distance $c(s) = (1 - \cos(\theta^* - \theta))/2$

Appendix G – Per-Test Error Bars (95% Binomial CIs)



Top: PPO-PBRS (blue) vs PPO-Curriculum (orange) head-to-head – per-scene trial-level SR from 20 trials. Overlapping CIs on most scenes.

Bottom: All 4 models – the self-play variants are near-zero on all but the easiest scenes.

Green labels: position-only tests. **Purple labels:** position+rotation tests.



Appendix H – P82 Curriculum Trigger Bug

The original PPO-Curriculum trigger never activated because it gated on an unreachable metric.

Broken trigger (original):

If $\text{mean}(\text{per-push PosErr}) < 0.08 \text{ m}$ for ALL 50 consecutive iterations $\Rightarrow$ activate Phase 2

Why unreachable:

- Per-push PosErr is measured after *every* push, including early ones far from a freshly randomized goal (0.1–0.45 m away)
- The iteration-mean has a structural floor well above 0.08 m – even the *best* model's validation PosErr is 0.202 m
- 50-iteration AND gate: one noisy iteration resets the streak

Consequence: $w_{\text{rot}} = 0.0$ permanently – rotation received zero reward signal. Model B (the pre-P82 version) was effectively position-only PPO.

Fix (P82): gate on *episodic position-only* $SR \geq 0.5$ over a 20-iteration lookback. This is a “position is solved” signal, not a per-push mean. Curriculum now activates

Appendix I – BobPenalty Fail-Fast Equilibrium



Model I (TASP-dPose-BobPenalty) adds a Bob time penalty $R_B += -\gamma_{sp} \cdot t_B$ on *all* phase ends, including catastrophes.

Why it creates a degenerate equilibrium:

Trajectory	PBRS + Sparse	Penalty ($0.5 \times t_B$)	Total return
Fail on push 1	-10.5	-0.5	-11.0
Fail on push 10	-15.0	-5.0	-20.0
Succeed push 5	+10.0	-2.5	+7.5

Mechanism:

1. PPO's gradient pushes Bob toward higher returns – “fail on push 1” (-11.0) beats “try for 10 pushes and fail” (-20.0)
2. Since failing is easier than succeeding in contact-rich physics, the policy collapses to immediate failure
3. Alice's reward collapses simultaneously: $R_A = 0.5 \cdot \max(0, 1 - 5) = 0$ – she gets zero signal⁶⁷

Appendix J – Confounded Ablation Disaggregation



Self-play variants differ from single-agent PPO-PBRS on ≥ 7 axes simultaneously:

Variable	PPO-PBRS (80.0%)	Self-play (6.7–16.7%)
Agent count	1	2 (Alice + Bob)
Goal distribution	Fixed random	Adversarial, shifting
GoalEncoder	None	ϕ -MLP 8D latent
ABC buffer	None	Yes (ASP-dPose) / No (TASP-dPose)
Historical pool	None	Yes
Episode budget	15 pushes/agent	5 (Alice) + 10 (Bob)
PPO updates/iter	1	2 (Alice, then Bob)

Implication: the $4.8\times$ SR gap cannot be attributed to any single lever.

- TASP-dPose vs ASP-dPose isolates Alice’s reward type – TASP-dPose gains +10pp
- But this is a single 1-axis experiment within a 7-axis package change
- We present “self-play as implemented,” not “the curriculum mechanism in isolation”



Appendix K – Training Budget Comparison

All PBRs variants used identical budgets. The PPO-Baseline used more compute for a direct architectural comparison.

Model	Envs	Max Iter	Est. Push-Macros
PPO-Baseline (Fractional)	2,048	100,000	~3.07B
PPO-PBRs	528	3,000	~23.8M
PPO-Curriculum	528	3,000	~23.8M
ASP-dPose	528	3,000	~23.8M
TASP-dPose	528	3,000	~23.8M
TASP-dPose-BP	528	3,000	~23.8M

Caveats:

- PPO-Baseline at 2,048 envs received $\sim 129\times$ more push-macros than PBRs models
- Both converge to 80% validation SR – PBRs achieves this with $4\times$ fewer parallel envs and a fraction of the total push budget



Appendix L – Validation Protocol

All models evaluated under an identical protocol:

- **Scene set:** 30 held-out T-block scenes – 10 rotation-dominant (R_*), 10 position-only ($E_*/M_*/H_*$), 10 combined position+rotation
- **Trials:** 20 independent trials per scene (best-of-20); a scene is “solved” if *any* trial meets the gate
- **Success gate:** $\text{pos_err} < 0.05 \text{ m}$ **and** $\text{rot_err} < 0.2 \text{ rad}$ at trial termination
- **Push budget:** up to 30 pushes per trial (training uses 5–15)
- **Scene SR vs Trial SR:** scene SR counts scenes where $\geq 1/20$ trials pass; trial SR counts individual trial success rate

Disclosed limitations:

- **Single-seed validation:** each model evaluated once (one seed per checkpoint). Training SR variance exists across 5 chains (PPO-PBRS BestSR 0.081–0.110) but validation CIs are single-model.